

PREDICTING ATP MATCH OUTCOMES: A COMPARISON OF LOGISTIC REGRESSION AND RANDOM FOREST MODELS

Lappeenranta–Lahti University of Technology LUT

Bachelor's Programme in Business Analytics, Bachelor's thesis

2026

Arttu Sokka

Examiners: M.Sc. Matias Heikkinen

Abstract

Lappeenranta–Lahti University of Technology LUT

LUT Business School

Bachelor's Programme in Business Analytics

Arttu Sokka

Predicting ATP Match Outcomes: A Comparison of Logistic Regression and Random Forest Models

Bachelor's thesis

2026

43 pages, 11 figures and 7 tables

Examiners: M.Sc. Matias Heikkinen

Keywords: Tennis, Match outcome prediction, Machine learning, Logistic regression, Random forest, Feature engineering, Cross-validation

This thesis compares logistic regression and random forest models for predicting men's ATP tennis match outcomes. The study uses Jeff Sackmann's ATP match data from 2000 to 2019, containing over 60,000 match observations. The models are trained on engineered features based on prior research and evaluated using a rolling training-testing window. The models' hyperparameters are optimized using time-series cross-validation, and predictive performance is measured with accuracy, log-loss, and the Brier score. In addition to predictive performance, the study examines feature importance and presents ways to interpret individual features under multicollinear conditions.

The results show that logistic regression achieved slightly higher classification accuracy (0.683) than random forest (0.678). This slight advantage can mainly be explained by the informative feature design and the rolling training-testing evaluation procedure. The leave-one-feature-out (LOFO) analysis further showed that the surface-specific Elo rating was the least replaceable feature, highlighting the importance of performance-related features in predicting match outcomes. Overall, the results demonstrate that carefully designed predictive features can yield competitive predictions with a simple logistic regression model in a rolling forecast setting.

Tiivistelmä

Lappeenrannan–Lahden teknillinen yliopisto LUT

LUT-kauppakorkeakoulu

Kauppätieteiden koulutusohjelma

Arttu Sokka

ATP-tennisotteluiden lopputulosten ennustaminen: Vertailu logistisen regression ja satunnaismetsämallien välillä

Kandidaatintyö

2026

43 sivua, 11 kuvaa ja 7 taulukkoa

Tarkastajat: DI Matias Heikkinen

Avainsanat: Tennis, Ottelun lopputuloksen ennustaminen, Koneoppiminen, Logistinen regressio, Satunnaismetsä, Muuttujien muodostaminen, Ristiinvalidointi

Tämä tutkielma vertailee logistisen regressiomallin sekä satunnaismetsämallin (random forest) suorituskykyä miesten ATP-tennisotteluiden ennustamisessa. Tutkimus perustuu Jeff Sackmannin ATP-otteludataan vuosilta 2000–2019, joka sisältää yli 60 000 otteluhavaintoa. Mallit koulutetaan aiempiin tutkimuksiin perustuvilla muodostetuilla ennustemuuttujilla ja arvioidaan rullaavalla koulutus-testausikkunalla. Mallien hyperparametrit optimoidaan aikasarjallisella ristiinvalidoinnilla, ja ennustekykyä mitataan tarkkuudella sekä log-loss- ja Brier-arvoilla. Tutkimus tarkastelee myös merkittävimpiä ennustemuuttujia ja esittää tapoja tulkita yksittäisten muuttujien tärkeyttä multikollinearisissa olosuhteissa.

Tulokset osoittavat, että logistinen regressio saavutti hieman korkeamman luokittelutarkkuuden (0,683) kuin satunnaismetsä (0,678). Tämä pieni etu selittyy pääasiassa informatiivisella muuttujien muodostamisella sekä rullaavalla koulutus-testausmenetelmällä. Leave-one-feature-out-analyysi (LOFO) osoitti lisäksi, että alustakohtainen Elo-luokitus oli vähiten korvattava muuttuja, mikä korostaa pelaajan suorituskykyä kuvaavien muuttujien tärkeyttä ottelutulosten ennustamisessa. Kokonaisuudessaan tulokset osoittavat, että huolellisesti suunnitelluilla ennustemuuttujilla voidaan saavuttaa kilpailukykyisiä ennusteita yksinkertaisella logistisella regressiomallilla rullaavassa ennustamisympäristössä.

Declarations

Turnitin

The originality of this thesis has been reviewed with the Turnitin similarity checking service.

AI Usage

The author of the thesis, Arttu Sokka, used the following AI-tools during the preparation of the thesis:

1. ChatGPT
 - a. Purpose of use: Language refinements and conceptual sparring
 - b. Explanation of the use of the tool: ChatGPT was used to improve text clarity and phrasing, and to clarify theoretical concepts. The sparring was done following the guidance of the thesis examiner.
2. Grammarly
 - a. Purpose of use: Language refinement
 - b. Explanation of the use of the tool: Grammarly was used to check grammar, spelling, and sentence structure. All suggestions were reviewed and applied by the author.

Responsibility

The author, Arttu Sokka, takes full responsibility for the content of this thesis and has reviewed and edited the content generated by the possible use of AI tools.

Table of contents

1	Introduction	7
1.1	Study aim and questions	8
2	Methodology	10
2.1	Logistic regression	10
2.1.1	Regularization in logistic regression	12
2.1.2	Multicollinearity in logistic regression	13
2.2	Random forest	14
3	Experimental Design	19
3.1	Data collection and cleaning	19
3.2	Feature design	21
3.2.1	Data representation and leakage prevention	21
3.2.2	Base differential features	22
3.2.3	Elo ratings	23
3.2.4	Serve advantage	24
3.2.5	Completeness	25
3.2.6	Form difference	25
3.2.7	Selected features	25
3.3	Model training	26
3.3.1	Hyperparameter tuning	26
3.3.2	Rolling forecast window	27
3.4	Evaluation metrics	29
3.4.1	Accuracy	29
3.4.2	Log-loss	30
3.4.3	Brier score	30
4	Experimental Results	32
4.1	Model setup and hyperparameter values	32
4.2	Predictive performance of the models	33

4.3 Feature importance	36
5 Conclusions	40
References	42

1 Introduction

Tennis is one of the most followed competitive sports in the world, where the ATP Tour represents the highest level of men’s professional tennis. This popularity has led to the growing availability of structured match data, enabling the application of quantitative methods in tennis analytics. As more data has become available, predicting tennis match outcomes has also become a central application in sports analytics. However, this prediction remains a challenging problem, because player performance varies over time and match outcomes ultimately depend on the relative superiority of the players in a matchup.

The prediction problem is straightforward because the match result is binary: a player either wins or loses. At the same time, accurate prediction is inherently uncertain, as outcomes are affected by various interacting and context-dependent factors (Baumer, Matthews & Nguyen, 2023). Bozděch & Zháněl (2023) further describes this complex nature, suggesting that independent features alone rarely explain match outcomes well, while Klaassen & Magnus (2001) shows that point outcomes in tennis are not independent and identically distributed. These findings together suggest that match outcomes are a combined effect of player characteristics, recent and long-term performance, and tournament-specific factors.

Sports outcome prediction is commonly approached using historical and context-dependent information before the match is played (Horvat & Job, 2020). In tennis, this makes the prediction setting especially dependent on how effectively match information is incorporated into predictive features before model estimation. Kovalchik (2020) further highlights the role of performance-based features, such as extensions of traditional ranking systems, in outcome prediction. Consequently, predictive performance depends on how well pre-match information is summarized through feature construction.

This also makes model comparison meaningful in this context, where logistic regression and random forest are widely used approaches for binary prediction. Logistic regression is often valued for its stability, linear structure, and straightforward interpretation, whereas random forest is more flexible at capturing nonlinear relationships and feature interactions (Bunker & Thabtah, 2019). However, greater flexibility does not automatically guarantee better predictive performance. If the feature set is carefully engineered, a substantial part of the underlying data complexity may already be incorporated into the predictors before model estimation.

The model training design plays a central role in this comparison because player performance varies over time, and the relevance of historical information can therefore decline. However, the training process must be designed to prevent data leakage, also known as look-

ahead bias, so that predictions are based solely on information available before the match. A meaningful model comparison therefore requires a forecasting procedure that respects the temporal structure of the data and effectively prevents look-ahead bias.

Previous studies have examined the prediction of tennis match outcomes using various machine learning approaches (Wilkens, 2021; Bunker, Yeung, et al., 2024). Existing research suggests that predictive performance depends not only on model choice but also on how effectively pre-match information is incorporated into predictive features, particularly in performance-related features (Kovalchik, 2020; Angelini, Candila & De Angelis, 2022). However, it is less clear whether a flexible nonlinear model provides additional predictive value when ATP match outcomes are predicted in a more realistic out-of-sample setting with carefully engineered features and strict data leakage prevention (Wilkens, 2021; Bunker, Yeung, et al., 2024). In addition, previous discussion has focused mainly on comparative forecasting performance, while less attention has been paid to whether different models rely on different parts of the engineered feature set, especially when performance features generally contain overlapping information (Gao & Kowalczyk, 2021; Wilkens, 2021).

1.1 Study aim and questions

This thesis examines ATP men’s singles match outcome prediction using pre-match information. The aim of the study is to compare logistic regression and random forest in a realistic forecasting setting. To complement existing literature, the study constructs features that summarize complex data relationships into an informative feature set.

The study also presents an advanced model training approach that effectively prevents look-ahead bias. In addition to comparing predictive performance, the study examines whether the most important features differ across the models while accounting for the overlapping information among the engineered features.

This leads to the following primary research question:

RQ1: *How does predictive performance differ between random forest and logistic regression for ATP match outcomes?*

To complement the primary research question, the study addresses the following secondary research question:

RQ2: *Do the most important pre-match features differ between the models in ATP match outcome prediction?*

To answer these research questions, the study first constructs a cleaned pre-match dataset from ATP men's singles matches and represents the data with predictive features. Logistic regression and random forest are then trained and evaluated using the same forecasting procedure, which prevents the look-ahead bias and respects the temporal nature of the match outcome prediction problem. The models are compared using selected evaluation metrics, and the role of each feature is then examined by excluding it from the feature set in turn.

Within this framework, the study proceeds as follows. Chapter 2 describes the methodology used in the analysis. Chapter 3 presents the dataset, feature construction, model training procedure, and evaluation metrics. Chapter 4 reports the selected parameter values and presents empirical results. Finally, Chapter 5 summarizes the study's main findings and discusses possible directions for future research.

2 Methodology

This chapter presents the study's methodology for binary classification. The methodology describes the models' properties, including their strengths and limitations. It also outlines approaches for addressing these limitations. The methodology is divided into two subsections based on the selected models.

2.1 Logistic regression

Logistic regression is a statistical model for classification problems with a binary dependent variable. The model was originally introduced by David Cox (1958). To formalize this model, let the dependent variable satisfy

$$Y_i \in \{0, 1\}. \quad (1)$$

Since Y_i is a binary variable, its conditional distribution is modeled as Bernoulli with success probability p_i ,

$$Y_i | X_i \sim \text{Bernoulli}(p_i), \quad (2)$$

where $X_i \in \mathbb{R}^{k+1}$ denotes a vector of features including a constant term for observation i . Therefore, the conditional probability of success is defined as

$$p_i = P(Y_i = 1 | X_i). \quad (3)$$

The logit model is then specified through a linear predictor. The linear predictor is defined as

$$\eta_i = X_i^\top \beta, \quad (4)$$

where $\eta_i \in \mathbb{R}$ denotes the linear predictor and β is a column vector of unknown parameters. Let

$$\beta = (\beta_0, \beta_1, \dots, \beta_k)^\top, \quad (5)$$

where β_0 is the intercept and $\beta_1, \dots, \beta_k$ are slope coefficients corresponding to the features. Accordingly, η_i reaches the entire real line $(-\infty, \infty)$. Since p_i is restricted to the interval $[0, 1]$, it cannot be modeled directly as a linear function of the features. Instead, consider the odds of the event, defined as

$$\frac{p_i}{1 - p_i}, \quad (6)$$

where p_i is the probability of the event and $1 - p_i$ is its complement. However, the odds take values in $(0, \infty)$ and are therefore not compatible with the unrestricted range $(-\infty, \infty)$ of the linear predictor. Taking the logarithm of the odds transforms them to $(-\infty, \infty)$, making the

log-odds compatible with the linear predictor $\eta_i = X_i^\top \beta$. The model is therefore specified as

$$\log \left(\frac{p_i}{1 - p_i} \right) = X_i^\top \beta. \quad (7)$$

Finally, solving the equation for p_i yields the logistic function.

$$p_i = \frac{1}{1 + e^{-X_i^\top \beta}}. \quad (8)$$

Because Y_i is Bernoulli distributed, the probability of observing a single outcome of y_i can be defined as

$$P(Y_i = y_i | X_i, \beta) = p_i^{y_i} (1 - p_i)^{1 - y_i}, \quad (9)$$

which combines both Bernoulli outcomes for $y_i \in \{0, 1\}$. When $y_i = 1$, the expression simplifies to p_i , and when $y_i = 0$, it simplifies to $1 - p_i$. By assuming that the observations are independent, the likelihood function for a sample of n observations can be written as the product of the individual Bernoulli probabilities:

$$L(\beta) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1 - y_i}. \quad (10)$$

The likelihood function measures the probability of the observed data given the parameter vector β . The objective of the logit is to estimate the parameter values that maximize this probability; it is therefore called the *maximum likelihood estimator* (MLE). The objective function becomes

$$\hat{\beta} = \arg \max_{\beta} L(\beta). \quad (11)$$

However, maximizing the likelihood in product form is analytically inconvenient. This issue can be addressed by taking the natural logarithm of the likelihood function, which transforms the product of probabilities into a sum by the properties of logarithms, and greatly simplifies differentiation. The logarithm is a strictly increasing function, and therefore maximizing the likelihood is equivalent to maximizing the log-likelihood. The log-likelihood function is

$$\ell(\beta) = \sum_{i=1}^n [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]. \quad (12)$$

In practice, optimization problems are often approached by minimization. Therefore, estimating the MLE is equivalent to minimizing the negative log-likelihood function as follows

$$\hat{\beta} = \arg \min_{\beta} (-\ell(\beta)). \quad (13)$$

The (log-)likelihood is nonlinear in the parameters, and therefore no closed-form solution exists, unlike in ordinary least squares (OLS) regression. Consequently, the estimator $\hat{\beta}$ must be computed using numerical optimization methods because the likelihood equations cannot be solved analytically. The interpretation of the coefficients in the logit also differs from that in OLS. In the logit model, the log-odds are specified as a linear function of the features. Because of the linearity, differentiating with respect to X_{ik} leaves only the corresponding coefficients β_k

$$\frac{\partial}{\partial X_{ik}} \log \left(\frac{p_i}{1 - p_i} \right) = \frac{\partial}{\partial X_{ik}} (X_i^\top \beta) = \beta_k. \quad (14)$$

Therefore, each coefficient β_k represents the change in the log-odds of the outcome corresponding to a one-unit increase in X_{ik} , holding the other features constant. This follows from the linear predictor $X_i^\top \beta$. If X_{ik} increases by one unit, the linear predictor increases by β_k . This implies

$$\log \left(\frac{p_i^{\text{new}}}{1 - p_i^{\text{new}}} \right) = \log \left(\frac{p_i^{\text{old}}}{1 - p_i^{\text{old}}} \right) + \beta_k. \quad (15)$$

Exponentiating both sides yields

$$\frac{p_i^{\text{new}}}{1 - p_i^{\text{new}}} = e^{\beta_k} \frac{p_i^{\text{old}}}{1 - p_i^{\text{old}}}, \quad (16)$$

where e^{β_k} is the odds ratio. This means that the odds are multiplied by e^{β_k} when X_{ik} increases by one unit, while the other features remain constant.

2.1.1 Regularization in logistic regression

Although the MLE is consistent under standard conditions, it is prone to producing excessively large coefficient estimates, particularly when features are highly correlated or the outcome is nearly perfectly separable. In these cases, the likelihood function is maximized by increasing coefficient values at the expense of estimator stability. These inflated estimates increase the model's variance and make it sensitive to minor fluctuations in the data, ultimately weakening its out-of-sample predictive performance.

The reason for this behavior lies in the nature of MLE. The estimation procedure optimizes the parameter values that maximize the likelihood function in Equations (10) and (11) without imposing any restrictions on the magnitudes of the coefficients. Under perfect or near-perfect separation, for example, when the features almost perfectly predict the outcome, the likelihood continues to increase. Therefore, no finite MLE exists as some coefficients grow without bound.

In addition, when predictors contain overlapping information, different coefficient vectors can produce nearly identical values of the linear predictor $X\beta$. This makes the estimation problem ill-conditioned and reduces the stability of the coefficient estimates. This reflects a lack of injectivity in the mapping $\beta \rightarrow X\beta$, meaning the coefficient vector cannot be uniquely identified from the data (James et al., 2023).

As a consequence, the estimation problem becomes unstable. Thus, small changes in the data may lead to large changes in the estimated coefficients. This reduces the model's reliability and makes the parameter estimates sensitive to sampling variation. To address these issues, ridge regularization introduces an L_2 -penalty term on large coefficients; instead of minimizing the negative log-likelihood alone, ridge logit minimizes a penalized objective function, defined as

$$\hat{\beta} = \arg \min_{\beta} \left(-\ell(\beta) + \lambda \sum_{k=1}^p \beta_k^2 \right), \quad (17)$$

where $\lambda \geq 0$ is the regularization hyperparameter that controls the strength of the penalty term, and $\sum_{k=1}^p \beta_k^2 = \|\beta\|^2$, denoting the squared L_2 -norm of the coefficient vector. When $\lambda = 0$, the objective function simplifies to the standard MLE objective. For $\lambda > 0$, large coefficient values increase the penalty term, and the optimization procedure therefore favors smaller coefficients closer to zero. Consequently, shrinkage reduces variance and stabilizes the model, particularly when features are highly correlated.

2.1.2 Multicollinearity in logistic regression

Multicollinearity occurs when two or more features are highly correlated, meaning that one predictor can be expressed as a linear combination of others. In such cases, the features overlap, making it difficult to distinguish the individual effects of the variables. In practice, a correlation of 0.80 or higher among features is often used as a limit to identify strong multicollinearity (Nayem, Aziz & Kibria, 2025).

The primary consequence of multicollinearity is the uncertainty and unreliability of the estimated coefficients. This most typically leads to larger standard errors for the features and weaker statistical significance in the model. As a result, coefficient estimates become sensitive to small changes in the data, and the model's interpretability weakens. This is particularly important when the research focus is on identifying the most important predictors among the variables.

However, multicollinearity does not necessarily reduce the model's predictive accuracy. The model can still utilize shared information across correlated features and produce accurate predictions even when individual coefficient estimates are unstable. Therefore, mul-

multicollinearity is a limitation for interpreting feature effects rather than for predictive performance.

Multicollinearity affects the logit model through the linear predictor Equation (7), where the log-odds are expressed as a linear combination of the features. In the context of MLE, strong correlations in predictors make it difficult to uniquely determine the coefficient vector that maximizes the likelihood function. Therefore, the likelihood surface may flatten in certain directions, since different coefficient combinations can produce similar values of the linear predictor.

Multicollinearity can be addressed in several ways to maintain the logit model's statistical significance. First, if two or more variables share nearly identical information, one of them can be removed to reduce redundancy in the model. This essentially simplifies the specification and decreases correlation in the predictors. Second, correlated variables can be combined into a single compound variable if the variables are considered important for predictive performance. This reduces overlapping information and feature correlation while preserving the model's performance.

Third, multicollinearity can be treated using principal component analysis (PCA). PCA transforms correlated predictors into a smaller set of orthogonal linear combinations, called principal components (PCs). Because the PCs are orthogonal, they are uncorrelated by construction. This improves the numerical stability, but the resulting PCs are less individually interpretable than the original variables. Each component represents a linear combination of the original variables and can therefore be expressed in terms of the original features.

In addition, the impact of multicollinearity can be reduced with ridge regularization. Ridge penalizes large coefficients shown in Equation (17) and therefore stabilizes the linear predictor $X_i^\top \beta$ in such cases where predictors are strongly correlated. Consequently, the model becomes less sensitive to correlations, which improves its reliability.

2.2 Random forest

Decision trees were first introduced by Breiman et al. (1984), which form the foundation of the random forest model. Random forest is an ensemble learning method further developed by Breiman (2001) that constructs multiple decision trees for classification or regression tasks. These decision trees are commonly referred to as classification and regression trees (CART). As this study focuses on classification, the decision trees are hence referred to as classification trees.

A classification tree is a hierarchical model that consists of a root node, internal nodes, and

leaf nodes. First, the tree starts with the root node, which contains the entire dataset. From the root node, the tree is built iteratively with features that best split the data into more homogeneous groups. Nodes with child nodes are called internal nodes, whereas nodes that no longer split the data are called leaf nodes. Nodes with upper nodes are called child nodes. Figure 1 illustrates the structure of a classification tree using two abstract features (*Feature 1*, *Feature 2*) and a binary target variable with 50,000 observations. The figure also demonstrates that each leaf node corresponds to a predicted class based on the majority of observations within the node.

Classification Tree Model

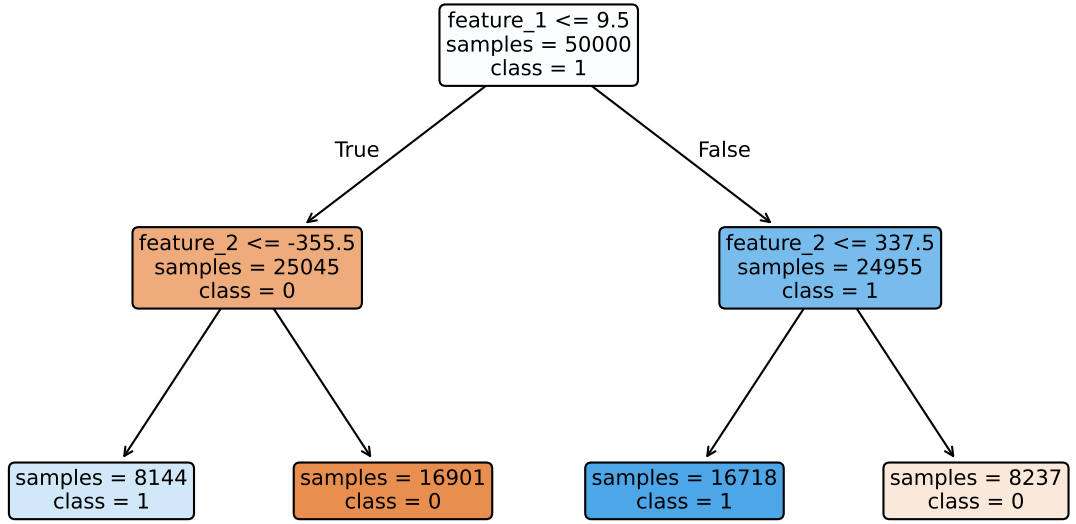


Figure 1: Structure of the classification tree.

The splitting procedure is repeated recursively for each internal node. In classification trees, the threshold for each split is determined by minimizing the impurity of the resulting internal nodes. Impurity is typically measured using the Gini index, where 0 indicates perfect homogeneity within a node. In binary classification, maximum heterogeneity corresponds to a Gini index value of 0.5 when the two outcome classes are evenly distributed. The algorithm divides the data into two subsets and computes the impurity of each resulting node. Finally, it selects the threshold that minimizes the weighted impurity.

The Gini impurity at node t is thus defined as

$$G(t) = 1 - \sum_{k=1}^K p_{k,t}^2, \quad (18)$$

in which K is the number of classes and $p_{k,t}$ represents the proportion of observations that belong to class k in node t . The Gini index measures the expected misclassification probability if an observation were randomly labeled according to the class distribution in the node t . The impurity is computed to evaluate the quality of a potential threshold. The classification tree's overall impurity is therefore calculated as the weighted average of the child nodes' impurities:

$$G_{\text{split}} = \frac{|D_L|}{|D_t|} G(L) + \frac{|D_R|}{|D_t|} G(R). \quad (19)$$

D_t denotes the set of observations in the node t , while D_L and D_R represent the subsets of observations to the left and right child nodes, respectively. The impurity of each child node is weighted by its proportion of the total observations in D_t , and these weighted values are summed to calculate the overall impurity of the split.

When more features are included, the algorithm iterates over all candidate features X_j and threshold values s . The weighted Gini impurity is then computed for each pair (X_j, s) . The objective is to select the feature and threshold that minimize this weighted impurity. Thus, the objective function of the classification tree can be expressed as

$$\arg \min_{X_j, s} G_{\text{split}}(X_j, s). \quad (20)$$

An individual tree is therefore constructed as follows. First, the algorithm evaluates all candidate feature-threshold pairs (X_j, s) and selects the combination that minimizes the weighted impurity. This split divides the data into two child nodes. The same procedure is then repeated for each child node using only the observations assigned to that node. As illustrated in Figure 2, this represents the general construction principle of a classification tree, where every node is defined by the locally optimal feature-threshold pair and different features may be selected at different stages of the construction. While this construction allows an individual tree to adapt flexibly to the training data, it also makes the tree sensitive to small variations in the data.

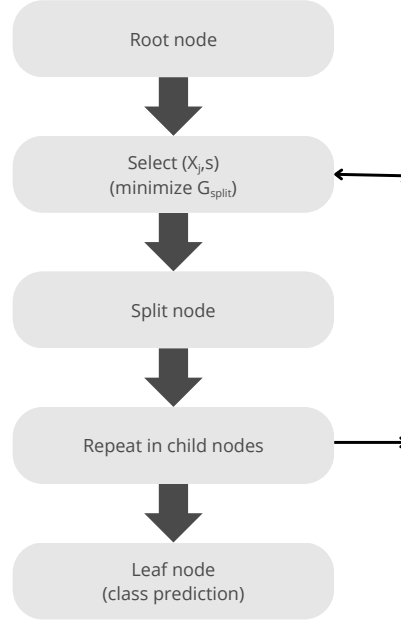


Figure 2: General construction of a classification tree using locally optimal feature-threshold splits.

Random forest constructs an ensemble of classification trees using a technique known as bootstrap aggregation, or bagging. The model generates bootstrap samples by sampling observations with replacement from the original training data, and trains separate trees on each sample. Each bootstrap sample has the same size as the original dataset, where some observations appear multiple times while others may not appear at all. Consequently, each tree is trained on a different resampled version of the data, thereby diversifying the trees and reducing the ensemble's overall variance.

Bagging effectively diversifies the training data for each tree; random forest further enhances this diversity by introducing feature randomness in the tree construction process. While standard bagging considers all available predictors at every split, the random forest algorithm considers only a random subset of features as split candidates at each node. This procedure prevents the few most dominant predictors from imposing the structure of the forest. In addition, this ensures the ensemble becomes highly decorrelated, as the individual trees use different features. Consequently, when diverse trees are aggregated into an ensemble model, the individual errors of trees tend to offset one another. This leads to a significant reduction in the ensemble model's expected variance, effectively addressing the bias-variance trade-off.

Let $\bar{X}$ denote the average prediction of the ensemble. Thus, the expected variance of the ensemble model is

$$\text{Var}(\bar{X}) = \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2, \quad (21)$$

where ρ is the correlation between two individual trees, σ^2 is the variance of an individual tree, and B is the number of random trees in the forest (Hastie, Tibshirani & Friedman, 2009). As the number of individual trees increases, the term $\frac{1-\rho}{B}\sigma^2$ approaches zero. Thus, the minimum achievable variance is limited by the correlation between the trees. The algorithm thereby reduces overall variance by minimizing ρ , resulting in a model variance lower than achievable with standard bagging alone.

The random forest incorporates specific hyperparameters that are essential to its ensemble structure and predictive stability. The most significant of these are the number of trees in the forest, the size of the random feature subset at each node, the maximum depth of an individual tree, and the minimum number of observations required in every leaf node. When the size of the random feature subset is limited, the algorithm effectively reduces the pairwise correlation ρ between trees, thereby helping prevent overfitting. Simultaneously, increasing the number of trees in the forest stabilizes the model's predictions and reduces the overall variance.

The maximum depth of an individual tree and the minimum number of observations required in every leaf node further control the complexity of individual trees. Restrictions on these hyperparameters prevent excessively intricate tree structures and reduce overfitting. Although a larger ensemble generally improves model performance, selecting these hyperparameters requires careful balancing.

3 Experimental Design

This chapter describes the empirical setup that was used to train and evaluate the predictive models. The section is divided into four subsections. First, the dataset and the data pre-processing are introduced. Second, the feature engineering process is described based on previous studies. Third, the model training is presented. Finally, evaluation procedures are described.

3.1 Data collection and cleaning

The study uses the open-source ATP tennis repository by Jeff Sackmann, in which each observation corresponds to a single match (Sackmann, 2026). The raw dataset contains 61,732 ATP Tour matches during the selected sample period, 2000–2019. The dataset includes tournament information, player characteristics and attributes, match outcomes, and selected in-match statistics. Table 1 summarizes the variables in the raw dataset.

Table 1: Variables in the ATP match dataset (based on official data dictionary).

Feature	Description
<i>Tournament information</i>	
tourney_id	Unique tournament identifier
tourney_name	Tournament name
surface	Court surface (Hard, Clay, Grass, Carpet)
draw_size	Number of players in the tournament draw
tourney_level	Tournament category
tourney_date	Tournament start date (YYYYMMDD)
match_num	Match identifier within tournament
<i>Player information</i>	
winner_id, loser_id	Unique player identifier
winner_name, loser_name	Player name
winner_hand, loser_hand	Dominant hand (R=right, L=left, U=unknown, A=ambidextrous)
winner_ht, loser_ht	Height in centimeters
winner_ioc, loser_ioc	Three-character country code
winner_age, loser_age	Age at tournament date in years
winner_seed, loser_seed	Player seed in tournament draw
winner_entry, loser_entry	Tournament entry category
<i>Match information</i>	
score	Final match score
best_of	Number of sets for the match (3 or 5)
round	Tournament round
minutes	Match duration in minutes
<i>Match statistics</i>	
w_ace, l_ace	Number of aces
w_df, l_df	Number of double faults
w_svpt, l_svpt	Number of service points played
w_1stIn, l_1stIn	Number of first serves made
w_1stWon, l_1stWon	First-serve points won
w_2ndWon, l_2ndWon	Second-serve points won
w_SvGms, l_SvGms	Service games played
w_bpSaved, l_bpSaved	Number of break points saved
w_bpFaced, l_bpFaced	Number of break points faced
<i>Ranking</i>	
winner_rank, loser_rank	ATP ranking at tournament date
winner_rank_points, loser_rank_points	ATP ranking points at tournament date

The full dataset spans the years from 1968 to 2024. However, this study uses only men's

singles matches from 2000 to 2019. The starting year 2000 was selected because data are more consistent than in the 1990s and earlier decades, and match records from 2000 onward contain fewer missing values and more standardized tournament information. The sample period ends in 2019, as the COVID-19 pandemic disrupted the ATP calendar and changed competitive circumstances from 2020 onwards.

The cleaning process, summarized in Figure 3, begins by removing matches played on carpet court surfaces due to their limited representation in the 2000s and their disappearance from the tour in the 2010s. Matches played in Davis Cup competitions were excluded because they differ structurally from ATP Tour tournaments. Davis Cup is a team-based national competition with a different competitive format, and the dataset contains a significant number of missing match statistics for these matches compared to other tournaments.

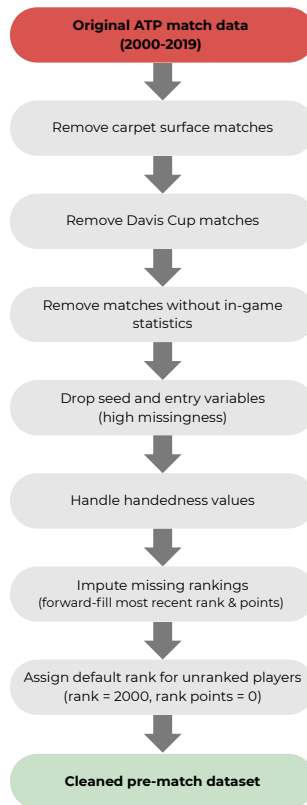


Figure 3: Data cleaning pipeline for the dataset.

Matches with missing in-game statistics were removed, as these variables were used to construct performance-based features. Seed and entry variables contained a substantial number of missing values and were therefore removed to ensure consistency. During the cleaning process, most observations with unknown handedness were either excluded or supplemented using information from the ATP website. As a result, one player remained with unknown handedness, and one player with ambidextrous handedness. These two players were therefore removed to ensure consistency; the handedness feature used in the forecasting was de-

defined as binary.

Missing ranking and ranking-points values are handled by forward-filling using each player's most recent ranking information. In addition, players without any ranking history are assigned a default rank of 2000 and 0 rank points, corresponding to the lowest possible ranking placement. This procedure substantially reduced the number of observations that would otherwise be removed due to missing ranking information. The cleaning pipeline yielded a dataset with no missing values and 53,099 ATP matches, compared to 61,732 observations in the raw dataset.

3.2 Feature design

Feature design presents the construction of the predictive variables used in the modeling stage. Feature engineering is the process of transforming raw data into informative features that improve the predictive performance of machine learning models (Ma et al., 2024). The first subsection addresses preventing the look-ahead bias, whereas the remaining subsections focus on constructing the predictive features used in the models.

3.2.1 Data representation and leakage prevention

The dataset records each match from a winner-loser perspective, where all player-specific variables are separated into *winner_** and *loser_** columns based on the match outcome. While this structure is suitable for descriptive analysis, it is structurally asymmetrical in a predictive modeling context because the winner-related variables always correspond to the positive match outcome, leading to bias in the model's training and testing.

To avoid this issue, players in each match were randomly assigned to *Player 1* and *Player 2* format before feature construction. In this representation, the target value is 1 if *Player 1* wins the match and 0 otherwise. Therefore, the outcome prediction problem in this study is to predict the match result from Player 1's perspective. This transformation removes the structural asymmetry in the data and ensures that the prediction problem represents a balanced comparison between two players.

Hence, all the following features are expressed as randomized player-pair comparison rather than a winner-loser representation. Consequently, the model learns relationships between the two competitors' characteristics rather than relying on the structural bias in the raw dataset. The even distribution of the target variable after randomization is illustrated in Figure 4.

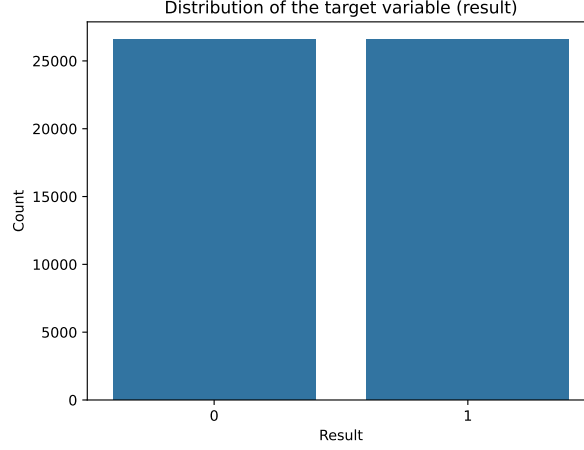


Figure 4: Distribution of the binary target variable after player randomization.

Moreover, data leakage occurs when information from future observations unintentionally influences the training of the predictive model. The primary consequence of such leakage is the model’s unrealistically optimistic prediction performance. The dataset was therefore arranged in chronological order by tournament date. Therefore, earlier matches always precede later observations in the data. Historical performance features were constructed using information available before the match, ensuring a realistic pre-match prediction setting.

3.2.2 Base differential features

The competitive relationship between players was represented using differential features. The study does not predict match outcomes based on players’ individual *ranking*, *age*, or *height* values, but rather by how these characteristics compare between the two competitors. Therefore, all predictive features, except player handedness, were expressed as differences between the two players in the matchup (Buhamra, Groll & Brunner, 2024). This not only measures the relative advantage of one player over the other but also aligns with the binary classification setting of this study. Accordingly, base differential features: *ranking*, *rank points*, *age*, and *height* were transformed into base differential features. The term *base* indicates that the variables were already presented in the raw dataset.

Let *player i* and *player j* denote *Player 1* and *Player 2* in a given match at time *t*, respectively, where *player i* is randomly assigned in each match. Thus, the differential feature is

$$\Delta Z_t = Z_{i,t} - Z_{j,t}, \quad (22)$$

where $Z_{i,t}$ and $Z_{j,t}$ denote the pre-match values of feature *Z* for players *i* and *j*. If the order of the players is swapped, the differential feature becomes $-\Delta Z_t$. This symmetry ensures that the models learn differences in matchups between players rather than players’ labels.

In addition to differential features, a binary *handedness matchup* feature was included as a relative attribute. The feature takes the value 1 if the players in the match have different dominant hands and 0 otherwise. Although handedness could be encoded in more detailed categories that cover all handedness options, it would likely add only limited additional information.

3.2.3 Elo ratings

The Elo rating system was developed by Arpad Elo (1978) to estimate the relative strength of chess players and has since been widely applied in sports. In this study, Elo ratings were computed sequentially after every match.

The standard Elo model is a two-step algorithm. First, the estimation computes the probability of player i defeating player j at time t . Therefore, the expected winning probability is

$$\hat{P}_{ij,t} = \frac{1}{1 + 10^{-\frac{Elo_{i,t} - Elo_{j,t}}{400}}}, \quad (23)$$

where $Elo_{i,t}$ and $Elo_{j,t}$ denote the players' respective Elo ratings at time t . The constant 400 determines how rating differences are translated into win probabilities in the standard Elo formulation (Elo, 1978), which has also been used in tennis Elo applications (Kovalchik, 2020).

After the match, player i 's Elo rating is updated as

$$Elo_{i,t+1} = Elo_{i,t} + K (W_{ij,t} - \hat{P}_{ij,t}), \quad (24)$$

where K is a constant learning parameter that determines how much a player's rating changes after a match, and $W_{ij,t}$ is a match result indicator, taking the value 1 for a win and 0 for a loss. The updated Elo rating is therefore obtained by adjusting the previous rating by the difference between the actual match outcome and the expected win probability, which is multiplied by the parameter K .

The Elo rating system can be interpreted as an estimate of players' latent competitive abilities over time. While the *rank points* feature is based on accumulated points over fixed time windows (e.g., seasons), Elo provides a continuously updating measure of player performance. This principle allows the Elo system to update player performances with a self-correcting mechanism (Kovalchik, 2020).

All players were assigned an initial Elo rating of 1500, which is a common starting value in Elo rating systems (Sackmann, 2019). Moreover, this study uses $K = 32$ as the learning

parameter, which has often been used as a fixed value in Elo-based tennis prediction literature (Bunker, Yeung, et al., 2024).

Elo ratings do not have fixed theoretical lower or upper bounds because the rating scale is open-ended, and rating differences are meaningful mainly through the probability calculation (Elo, 1978). Elo ratings were transformed into a differential feature defined as

$$\Delta Elo_t = Elo_{i,t} - Elo_{j,t}, \quad (25)$$

where $Elo_{i,t}$ and $Elo_{j,t}$ denote the pre-match Elo ratings of player i and player j at time t . This representation is also consistent with the Elo framework, where the updated rating is ultimately computed based on rating differences between players.

Elo rating captures players' overall competitive strength well. However, it does not account for the court surface, an essential factor in the match outcome. Players usually perform differently on hard, clay, and grass courts due to their playing styles, rally lengths, and movement patterns. To incorporate this dependence into the models, surface-specific Elo ratings were constructed. The surface-specific Elo ratings were therefore updated only using matches played on the corresponding surface. This setting allows the model to capture surface-dependent players' abilities that the standard Elo may not fully recognize.

Consistent with the standard Elo representation, the surface-specific Elo feature is defined as

$$\Delta SElo_t = SElo_{i,t} - SElo_{j,t}, \quad (26)$$

where $SElo_{i,t}$ and $SElo_{j,t}$ denote the pre-match surface-specific Elo ratings of player i and player j at time t .

3.2.4 Serve advantage

The serve performance is a key determinant of match outcomes. Players who serve better than their opponents are more likely to control and ultimately decide the matches. To incorporate this aspect of the playing performance into predictive models, a rolling serve advantage feature was constructed based on players' historical serving and returning performance over their most recent 20 matches.

A traditional approach to evaluating serve performance is to analyze players' serve and return performance percentages separately. However, considering these independently does not account for the interaction between players' serve and return performance. Therefore, this leads to a loss of matchup information.

To account for both players' serve performance and their interaction, this study uses the serve advantage feature introduced by Sipko (2015). The serve advantage is the following

$$\Delta Serv_t = (SPW_{i,t} - RPW_{i,t}) - (SPW_{j,t} - RPW_{j,t}), \quad (27)$$

where $SPW_{i,t}$ and $SPW_{j,t}$ denote the proportion of *service points won* out of all service points played by player i and player j in their past 20 matches. $RPW_{i,t}$ and $RPW_{j,t}$ denote the proportion of *return points won* out of all return points played by *player i* and *player j* in their past 20 matches.

3.2.5 Completeness

Similar to the serve advantage feature, top players typically perform strongly in both serving and returning; success is not determined by serving ability or return performance alone, but by the combination of both. To measure this balanced performance, this study utilizes the completeness feature introduced by Sipko (2015). The feature is defined as the difference between the players' products of service points won and return points won, computed over their past 20 matches.

$$\Delta Comp_t = (SPW_{i,t} * RPW_{i,t}) - (SPW_{j,t} * RPW_{j,t}). \quad (28)$$

The feature's structure ensures that a player achieves a high completeness score only if they perform well in both serving and returning.

3.2.6 Form difference

A player's recent performance, often referred to as form, indirectly captures short-term factors such as injuries and other temporary changes in a player's performance. The form feature is computed as the difference between the players' rolling win percentages over their previous 20 matches, excluding the current match to avoid data leakage. Therefore, the feature is

$$\Delta Form_t = Form_{i,t} - Form_{j,t}, \quad (29)$$

where $Form_{i,t}$ and $Form_{j,t}$ denote the average winning percentage of *player i* and *player j* over their past 20 matches.

3.2.7 Selected features

Finally, Table 2 presents the final set of constructed features and their summary statistics. The candidate features *height difference* and *handedness matchup* were excluded from the final feature set because they did not improve predictive performance in either model.

Table 2: Summary statistics of the final set of constructed features.

Feature	Type	Min	Max	Mean	Std	Missing values
<i>Player characteristics</i>						
Age difference	Numeric	-20.9	25.9	0.0	5.2	0
<i>Ranking and rating</i>						
Rank difference	Numeric	-1991	1998	0.3	146.2	0
Rank points difference	Numeric	-16304	16641	2.8	2149.1	0
Elo difference	Numeric	-904.6	935.2	-0.1	196.1	0
Surface Elo difference	Numeric	-834.9	806.2	-0.1	178.5	0
<i>Recent performance</i>						
Completeness	Numeric	-0.3	0.3	0.0	0.1	0
Serve advantage	Numeric	-0.3	0.3	0.0	0.1	0
Form difference	Numeric	-1	1	0.0	0.3	0
<i>Target variable</i>						
Result	Binary (1 = <i>player i</i> wins)	0	1	0.5	0.5	0

3.3 Model training

The section describes how the models were trained and how the models' hyperparameters were evaluated and selected. The procedures are designed to respect the temporal structure of the data and to adapt to real-world prediction conditions. Models are trained and tested using Python's *scikit-learn* library.

3.3.1 Hyperparameter tuning

Hyperparameter tuning was applied to both models to improve predictive performance and ensure a fair comparison. Logistic ridge regression includes an L2-regularization hyperparameter that controls the penalty term's strength. In *scikit-learn*, this hyperparameter is expressed as $C = 1/\lambda$, so smaller values of C correspond to stronger regularization. Logistic ridge regression was implemented within a preprocessing pipeline where features were standardized using *StandardScaler* before the estimation. *StandardScaler* sets the features to zero mean and scales them to unit variance. Since regularization is applied, standardization ensures that features measured on different scales are treated equally by the penalty term.

The random forest model includes several hyperparameters that control the ensemble's structure and complexity. This study includes the number of trees in the forest, the maximum tree depth, the minimum number of observations per leaf node, and the number of features considered at each split. Unlike logistic ridge regression, random forest does not require feature standardization because classification trees split the data based on threshold values rather than distances between observations.

To select the optimal hyperparameter values, both models were tuned using expanding time-series cross-validation (TSCV). TSCV is a forward-looking validation method in which the training data always precedes the validation data in time. More specifically, the optimal hyperparameter values are selected based on an expanding training window within the TSCV procedure. As the amount of training data increases, the model tends to favor different parameter values over time. The expanding TSCV is particularly suitable for this prediction setting, as hyperparameters are validated using an increasing amount of historical information while preserving the temporal order of the data. Figure 5 demonstrates an example of the expanding TSCV procedure with two folds within a single rolling training window.

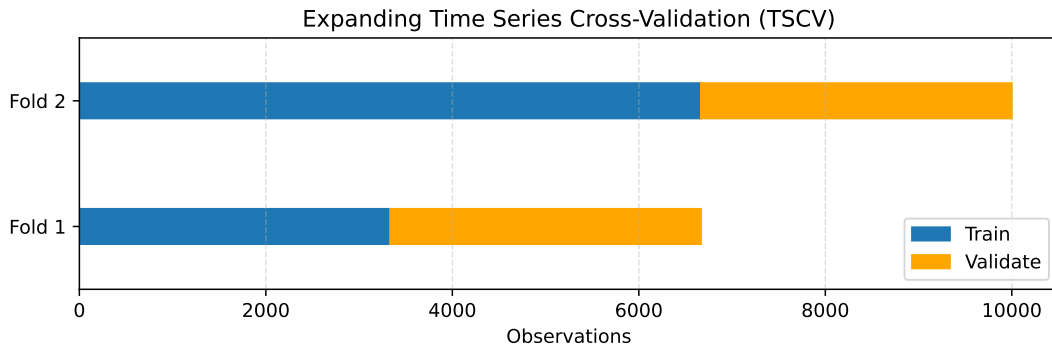


Figure 5: Expanding Time Series Cross-Validation within one training window.

Hyperparameter selection was performed through grid search, which evaluates all predefined combinations of hyperparameter values and selects the configuration that achieves the best validation performance. The optimal selection was determined using validation log-loss metrics. In *scikit-learn*, grid search maximizes the negative log-loss, which corresponds to minimizing log-loss (scikit-learn, 2023a). The log-loss metric is introduced in the next section. Ultimately, the selected hyperparameter values are summarized in the *Experimental Results* section.

3.3.2 Rolling forecast window

The models were trained and tested using an out-of-sample forecast window, a forecasting method in which the model is trained on past observations and tested on subsequent, unseen observations. The split was conducted using a rolling forecast window with a fixed training and testing period. This approach, together with the data’s chronological order, avoids look-ahead bias and enables realistic prediction conditions.

The models are trained using a fixed training period and then evaluated on unseen test data. After the prediction step, both the training and test windows are moved forward by a fixed step, and the same procedure is repeated until the end of the dataset. Thus, the models

always utilize only the information available at the time of prediction and do not have access to future observations. This produces a series of out-of-sample forecasts over the full sample period. Figure 6 presents an example of the rolling forecast window showing the first three train-test splits.

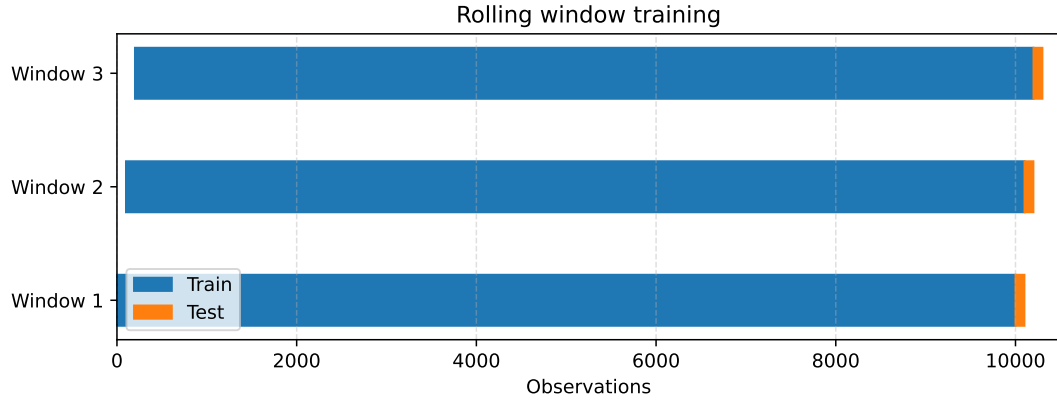


Figure 6: Example of a rolling forecast window.

The rolling forecast window is particularly suitable in the present setting because player performance changes over time due to injuries, aging, and other changes in form. Consequently, a fixed-length rolling window places greater weight on recent observations and gradually discards the older match data. The rolling design therefore provides a balanced compromise between recent and older information in the training sample. In addition, the rolling forecast window is computationally more efficient than an expanding window, because the size of the training sample remains fixed over time. This is particularly relevant in modern ML models, where continuously validated hyperparameters and model re-estimation substantially increase computational time.

Finally, the predicted probabilities, predicted outcomes, and actual match results were stored after each iteration and later used to evaluate the models' predictive performance using the selected evaluation metrics. The model's rolling training process is illustrated in Figure 7.

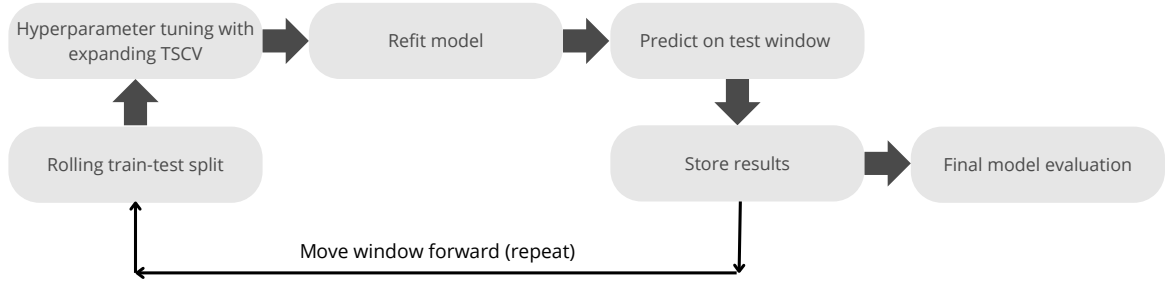


Figure 7: Rolling model training and evaluation procedure.

3.4 Evaluation metrics

Models' predictive performance is evaluated using three metrics: accuracy, log-loss, and Brier score. The metrics are selected to assess model performance from different perspectives and ensure a comprehensive evaluation of binary match outcome prediction.

3.4.1 Accuracy

The confusion matrix, shown in Figure 8, summarizes the classification performance of the model by comparing predicted outcomes with actual outcomes. It divides the predictions into four possible categories: *true positives* (TP), *true negatives* (TN), *false positives* (FP), and *false negatives* (FN). TP and TN represent correctly classified observations, whereas FP and FN represent classification errors. Consistent with the feature design, *player i*'s wins are treated as positive outcomes and *player i*'s losses as negative outcomes. Accuracy is

Confusion Matrix

Predicted Values	Positive	Negative
	TP	FP
Negative	FN	TN
	Positive	Negative
	Actual Values	

Figure 8: Structure of the confusion matrix.

derived from the confusion matrix and measures the overall proportion of correctly classified

outcomes. Thus, accuracy is calculated as

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad (30)$$

where $TP + TN + FP + FN = N$ denotes the total number of observations. Accuracy provides a simple, intuitive measure of model classification performance by indicating the overall correctness of predictions. The metric can be interpreted straightforwardly, because the match outcome classes are evenly distributed and accuracy therefore does not favor one class over the other.

3.4.2 Log-loss

Log-loss (logistic loss) evaluates the quality of the predicted probabilities, whereas accuracy measures the correctness of the predicted class labels. Log-loss also accounts for the model's confidence in its predictions, making it useful when the objective is to evaluate reliable probability estimates rather than only correct class labels.

The metric penalizes predictions according to the distance between the predicted probability and the actual outcome. The penalty is small if the model assigns a high probability to the event that actually occurs, and large when it assigns a high probability to the wrong outcome. Accordingly, log-loss rewards confident correct predictions, but heavily penalizes confident incorrect predictions. Therefore, under ideal conditions, the model achieves a log-loss of 0.

Log-loss is based on the same expression as the log-likelihood function used in logistic regression, but its role differs between model training and evaluation. As an evaluation metric, log-loss measures the negative average log-likelihood per observation and therefore assesses how closely the predicted probabilities match the actual outcomes. In *scikit-learn*, log-loss is defined as

$$\text{Log-loss} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)], \quad (31)$$

which is consistent with the estimation procedure presented in Equations (11)–(13) (*scikit-learn*, 2023c). A lower log-loss value indicates that the predicted probabilities are more closely aligned with the actual outcomes, whereas a higher value suggests weaker probabilistic predictions.

3.4.3 Brier score

Brier score is an evaluation metric for probabilistic predictions in binary classification. Similar to log-loss, the Brier score evaluates the quality of predicted probabilities, measuring

how close the predicted probabilities are to the actual outcomes on average.

More specifically, the Brier score is defined as the mean squared difference between the predicted probability of the outcome and the binary actual outcome. The penalty is therefore small when the predicted probability is close to the actual outcome. The Brier score for a sample of n observations is defined as

$$\text{Brier Score} = \frac{1}{n} \sum_{i=1}^n (p_i - y_i)^2, \quad (32)$$

where p_i denotes the predicted probability and y_i the actual outcome (scikit-learn, 2023b). When *player i* wins the match, the squared prediction error is $(p_i - 1)^2$, and if *player i* loses, the squared prediction error simplifies to p_i^2 . Thus, the Brier score is the mean of these squared prediction errors over all observations. The metric provides a more intuitive and easily interpretable measure of probabilistic accuracy than log-loss, making it a meaningful complement to the evaluation metrics used in this study.

4 Experimental Results

This chapter presents the study’s empirical results. The chapter is divided into three subsections. First, model setup and hyperparameter values are presented. Second, the predictive performance of logistic regression and random forest is compared. Finally, the role of individual features in the models’ predictions is examined.

4.1 Model setup and hyperparameter values

The model training parameters were chosen to balance predictive realism, temporal validity, and computational efficiency. Both models were evaluated using the same rolling out-of-sample forecasting, with hyperparameters selected within each training window via expanding time-series cross-validation (TSCV). In this setup, the training window size was fixed at 10,000 observations, corresponding to approximately four years of ATP match data, and the step size was set to 100 observations. The window and step sizes ensured that models produced frequent predictions without being trained on outdated information.

To reduce computational cost, expanding TSCV was limited to two validation folds per window. In the first fold, the model was trained on the earliest observations and validated on the subsequent period. In the second fold, the training window was expanded to include both the original training and validation data, and the model was validated on the remaining observations. This procedure produced two log-loss values for each hyperparameter configuration, which were averaged across folds. The configuration with the lowest average log-loss was then selected for refitting the model on the full training window.

Optimal hyperparameter values varied over time. Logistic regression included four selectable values for the regularization parameter C (3.3.1), while random forest included two alternatives for the number of trees, maximum tree depth, minimum leaf size, and the number of features considered at each split. Table 3 summarizes the hyperparameter grids and their proportions in percentages across all iterations.

Table 3: Hyperparameter value selection and proportions.

Model	Hyperparameter	Search grid (proportion in %)
Logistic Regression	C	0.01 (15.3), 0.1 (63.5), 1 (7.7), 10 (13.5)
Random Forest	n_estimators	150 (28.5), 200 (71.5)
	max_depth	6 (93.7), 8 (6.3)
	min_samples_leaf	5 (34.8), 10 (65.2)
	max_features	3 (99.8), None (0.2)

Logistic regression generally favored stronger ridge regularization, indicating that coefficient shrinkage improved predictive performance and reduced overfitting. However, the regularization level was not constant over time, as larger values of C were also selected in some iterations.

Random forest favored more concentrated hyperparameter values. The model favored shallower trees, indicating that limiting tree depth improved generalization. Similarly, a larger number of trees was chosen more frequently, consistent with their role in reducing ensemble variance. Selected hyperparameters suggest that random forest restricted the model complexity, likely due to the relatively small training sample available at each iteration. Furthermore, limiting the number of features considered at each split appeared to improve performance by reducing correlation between trees.

Models showed systematic structural preferences throughout the sample period, such as strong regularization in logistic regression and restricted tree complexity in random forest, where the expanding TSCV procedure further strengthened the reliability of these choices. Value distributions suggest that the model benefited more from stable model specifications than from unrestricted flexibility. Furthermore, adding selectable hyperparameter values would likely not have substantially increased the models' performance.

4.2 Predictive performance of the models

The confusion matrices in Figure 9 show that logistic regression slightly outperformed random forest. Both models classified the outcome classes without systematic bias. However, logistic regression produced fewer misclassifications, particularly in classifying false positives.

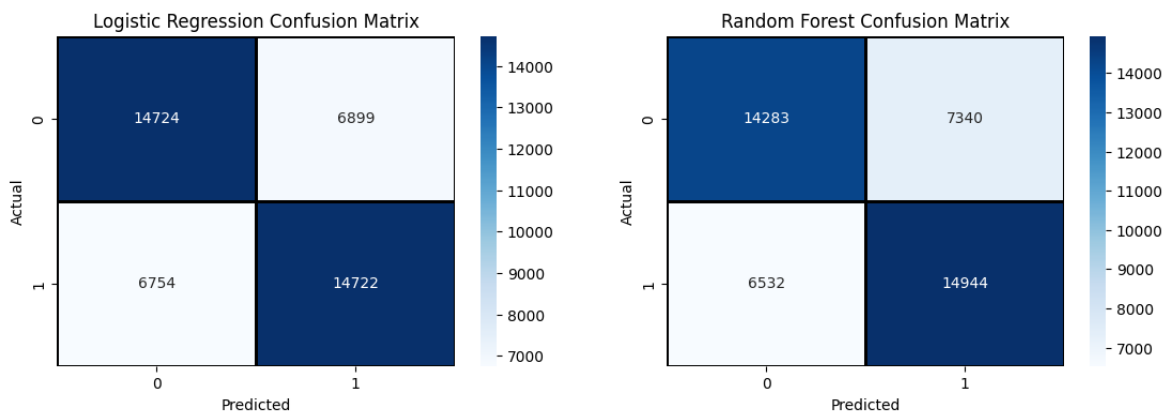


Figure 9: Confusion matrices of the models.

Table 4 summarizes the overall predictive performance. Since the outcome classes are evenly

distributed, the accuracy is computed from the confusion matrices as the proportion of correctly classified matches. Logistic regression achieved slightly higher accuracy than random forest, although the difference is small. As a reference, a classifier that predicted the higher-ranked player to win achieved an accuracy of 0.65, highlighting that the engineered features contributed slight but meaningful additional predictive information.

Table 4: Overall predictive performance of the models and the ranking baseline.

Model	Accuracy	Log-Loss	Brier Score
Logistic Regression	0.683	0.588	0.202
Random Forest	0.678	0.593	0.204
Ranking Baseline	0.650	—	—

The rolling forecast window, together with the moderate to strong regularization, likely helped logistic regression learn feature-outcome relationships more effectively. In contrast, random forest may not have benefited from its ensemble flexibility due to the smaller training data and relatively small feature set, which could have led to overfitting.

Additionally, log-loss and Brier score provide insight into the quality of the predicted probabilities, with lower values indicating better probabilistic predictions. Logistic regression produced probabilities that were slightly more closely aligned with the actual outcomes, while heavily penalizing confident incorrect predictions. The Brier score further suggests that the performance differences were due to logistic regression’s more consistent probability estimates rather than to random forest’s overconfident predictions. In addition, the feature engineering appears to have captured most of the data nonlinearity in a linearly usable form, allowing simple logistic regression to perform effectively.

The model differences were closely related to the training procedure. While the rolling window effectively captures players’ evolution over time, it limits the amount of training data available at each iteration. This may particularly affect random forest, as the model generally requires larger training samples to fully benefit from the ensemble’s structure. Logistic regression, in contrast, can produce accurate predictions even under limited data conditions, suggesting that the chosen training design favors more stable models.

The sample period from 2000 to 2019 is not excessively short, and tennis match data does not become too outdated over the 20-year sample period. This suggests that a larger training window could potentially be a compromise between realism and predictive performance. For instance, using an expanding window approach in model evaluation, as in the hyperpa-

parameter validation in TSCV, could allow models to learn more historical data and therefore produce more accurate predictions. However, the expanding window approach is computationally intensive, especially when hyperparameters are selected via TSCV, which limits its implementation in the study. Computational constraints may ultimately limit the random forest model’s ability to achieve its full predictive performance.

Models’ overall performance does not specify how predictions differ across skill levels. To examine this, predictions were grouped according to *player i*’s ATP ranking. Table 5 presents the prediction accuracy across the constructed ranking groups, where the weighted average of the groups’ accuracies corresponds to the model’s overall accuracy.

Table 5: Model accuracy by *player i*’s ranking group.

Ranking group	Observations	Logistic Regression	Random Forest
Rank 1–3	1710	0.842	0.839
Rank 4–10	3403	0.755	0.751
Rank 11–25	6311	0.684	0.680
Rank 26–50	9484	0.660	0.656
Rank 51–100	13342	0.661	0.656
Rank 101–2000	8849	0.680	0.673

Both models achieved their highest accuracy when *player i* belonged to the top 3, highlighting that a clear competitive advantage makes outcomes easier to predict. Accuracy declined as player rankings decreased, but remained relatively stable outside the top 10, indicating that high overall accuracy is particularly a result of correctly classified matches involving the very best players.

Because ranking groups do not capture how the balance of the matchup affected predictions, performance was additionally examined by grouping matches according to the absolute ranking difference between the players, as shown in Table 6.

Table 6: Model accuracy by absolute ranking difference.

Absolute ranking difference	Observations	Logistic Regression	Random Forest
0–10	6344	0.616	0.605
11–25	8336	0.635	0.626
26–50	10774	0.668	0.668
51–100	10162	0.723	0.721
101–250	5611	0.746	0.743
251+	1872	0.778	0.772

Models’ predictive performance increased systematically as the ranking difference became larger, demonstrating that both models perform best when there is a clear skill gap between players, particularly in matches where top-ranked players faced substantially weaker opponents. Logistic regression slightly outperformed random forest across all ranking-difference groups, except in the 26–50 range, suggesting that it captures skill differences more consistently with a limited amount of training data. Thus, the models’ additional predictive performance, compared to the ranking baseline, appears to be achieved particularly by correctly predicting matches in which top-ranked players faced substantially weaker opponents.

4.3 Feature importance

The feature importance analysis first examined model-specific importance measures. In logistic regression, importances were measured using standardized coefficients, whereas random forest feature importance was measured by the average decrease in Gini impurity across all tree splits. Figure 10 presents the averaged model measures across all rolling window iterations.

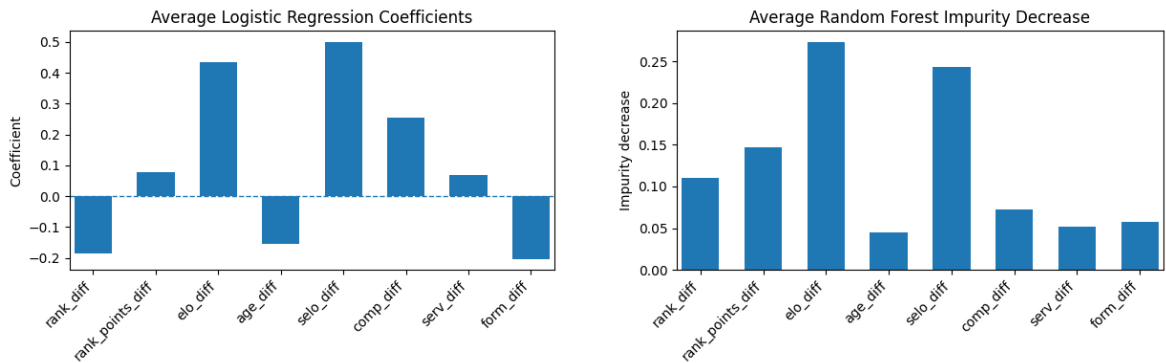


Figure 10: Feature importance of the models.

Figure 10 does not show great differences between the models. Both models used primar-

ily ranking- and rating-related information, particularly Elo-based features, while the role of player characteristics appeared smaller. *Rank points differences* were relatively more important in random forest than in logistic regression. However, Figure 11 illustrates that several performance-related features share overlapping information, so interpreting features using importance measurements is not straightforward in this study.

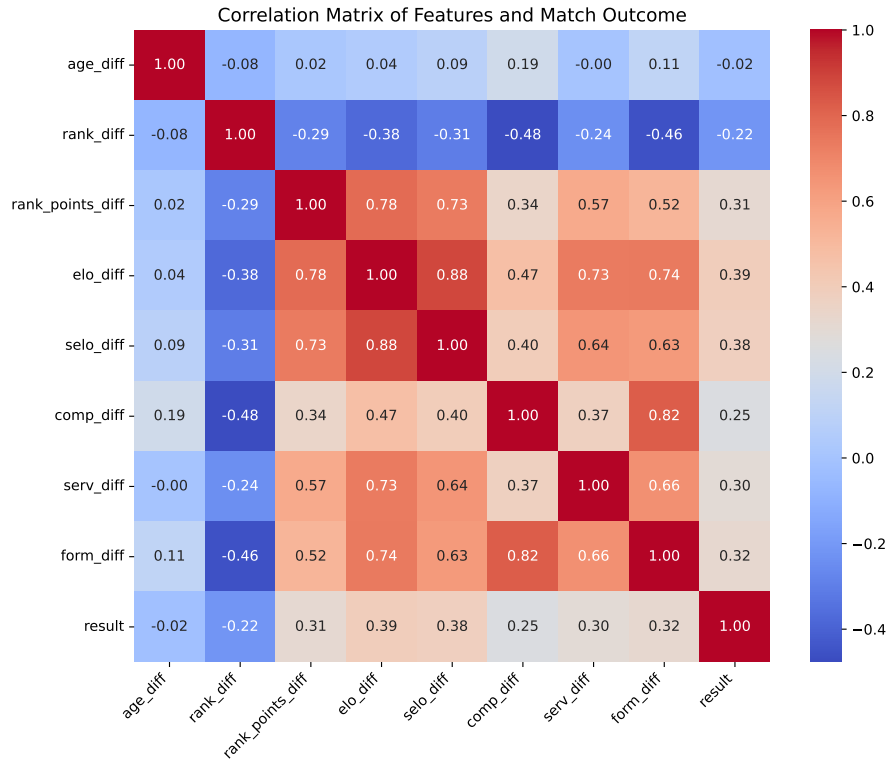


Figure 11: Correlation matrix of the used features and the match outcome.

Ridge regularization in logistic regression mitigates multicollinearity by shrinking coefficient estimates, but it does not remove dependence among correlated features. In random forest, multicollinearity is less problematic for predictive performance due to the ensemble structure, but it still complicates the interpretation of impurity decrease, since correlated features substitute for one another during node splitting.

The used feature set requires feature importance to be examined in alternative ways, one of which is to evaluate the replaceability of individual features in terms of prediction performance. Thus, this study includes a leave-one-feature-out (LOFO) analysis, in which each feature is removed from the feature set in turn, and the model is re-estimated without it. If removing a feature clearly decreases predictive performance, the feature contained information that the remaining features did not fully capture. On the other hand, if model performance does not decrease, the removed feature is likely to overlap with other features and could therefore be removed from the feature set. Table 7 presents the LOFO accuracies for logistic

regression and random forest.

Table 7: LOFO accuracy within the models.

Removed feature	Logistic Regression	Random Forest
None (full model)	0.683	0.678
<i>rank_diff</i>	0.681	0.678
<i>rank_points_diff</i>	0.683	0.677
<i>age_diff</i>	0.678	0.676
<i>elo_diff</i>	0.681	0.677
<i>selo_diff</i>	0.677	0.674
<i>comp_diff</i>	0.681	0.678
<i>serv_diff</i>	0.683	0.677
<i>form_diff</i>	0.682	0.677

The results show that removing individual features caused only minor changes in performance, even though small percentage differences correspond to hundreds of matches being correctly classified. This suggests that the models do not strongly depend on any single feature, but instead use partly overlapping information from ranking, rating, and recent performance features. Therefore, features whose removal did not decrease accuracy appeared to be replaceable by the remaining features, although they could alone explain match outcomes moderately. This is especially visible in *rank_points_diff* and *serve_diff* in logistic regression, and *rank_diff* and *comp_diff* in random forest. Nevertheless, these features were not removed from the final model evaluation, as the performance changes were not consistent across both models.

Removing *selo_diff* led to the clearest decrease in accuracy, which is consistent with previous findings on the importance of Elo-based applications in tennis prediction (Kovalchik, 2020). This indicates that the other features did not fully capture the performance difference across hard, clay, and grass courts. The removal of *age_diff* also reduced the accuracy in both models, although the effect was small. Age is a simple measure, but it still captures differences in players' physical development, experience, and career stage that are not fully represented by ranking and rating features. On the other hand, models may capture excluded *height_diff* in long-term performance trends, whereas handedness may have limited value, since most matches are played between players with the same dominant hand. In total, the results suggest that player characteristics played a smaller role than performance-based information, although the impact of age was greater in the LOFO analysis than in the measurements shown in Figure 10.

LOFO results indicate that each included aspect of the matchup contributed only limited additional predictive value. Together with the partly multicollinear feature set, this suggests that incorporating more diverse aspects of the game could make the feature set richer and less performance-driven. This could not only improve predictive performance but also reduce limitations in individual feature interpretability.

Figure 10 suggested that rating and ranking features would mainly explain the predictions, while LOFO reveals that their additional impact remains limited. The role of recent performance features appears to be larger in the LOFO than in the coefficients and impurity decreases indicated. However, strong conclusions should not be made from either approach alone. Since LOFO is based on accuracy, some feature effects may not fully appear in class predictions if the removed feature changes predicted probabilities without changing the predicted winner.

5 Conclusions

The main objective of this study was to compare the predictive performance of logistic regression and random forest models in a realistic ATP match prediction setting. The analysis used a cleaned dataset of more than 50,000 ATP men's singles matches in a realistic prediction setting, designed to carefully prevent look-ahead bias while maintaining computational feasibility. Logistic regression outperformed random forest in all selected evaluation metrics, producing slightly more accurate class predictions and probability estimates. Although the differences were small, they were systematic throughout the evaluation process. The results therefore suggest that a stable linear model is a more suitable choice under time and computational constraints, as the random forest's greater complexity did not provide a predictive advantage.

Furthermore, the constructed feature set also explains the findings. Most of the nonlinear match-affecting factors were already summarized into informative predictive features before model evaluation, indicating that logistic regression benefited more from the feature structure than random forest, although the model itself is linear. Both models' predictive performance is competitive compared to existing research (Bunker, Yeung, et al., 2024; Wilkens, 2021; Sipko, 2015). However, earlier research has often found that random forest models classify match outcomes more accurately (Gao & Kowalczyk, 2021; Wilkens, 2021). In this study, the slight advantage of logistic regression was likely related to the structure of the training-testing procedure. The expanding TSCV procedure and the model's forecasting window together favored a model that could provide accurate estimates with smaller training samples. Random forest, in contrast, generally requires larger training samples to fully benefit from its ensemble structure.

The study also examined the importance of the features used by the models, where logistic regression coefficients and random forest impurity decreases both suggested that the models mainly used ranking- and rating-related information. While the engineered feature set was substantially multicollinear, the LOFO analysis provided a more suitable interpretation under these conditions. Removing individual features resulted in only small changes in accuracy, indicating that ranking, rating, and recent-performance features were partially replaceable. Surface-specific Elo and age difference produced the clearest decreases when removed, suggesting that they captured information not fully represented by the remaining variables. Overall, the models relied mainly on performance-based information, while player characteristics had a more limited role.

The findings involve multiple limitations. First, although the feature set achieved high accuracy with relatively few features, its size and multicollinearity limited feature-level inter-

pretation. Future research could therefore construct a more diverse feature set, for example, replacing some multicollinear features with tournament context, player fatigue, travel effects, and additional court-specific performance measures. Second, future work could incorporate betting market odds to empirically test whether the models might outperform the betting market.

The results should also not be generalized too broadly. The study focuses on ATP men's singles matches from 2000 to 2019, and the findings are likely to differ in women's tennis, doubles, and lower-level tournaments. In addition, the study did not evaluate model performance during rapidly changing environments, such as the COVID-19 period. Therefore, the results only demonstrate strong predictive performance in this specific pre-match setting.

This thesis highlights the importance of informative feature construction for predicting ATP match outcomes. The results suggest that feature engineering explains prediction performance more than the model choice when using the presented prediction procedure. Under these conditions, the difference between logistic regression and random forest remained small, implying that accurate and computationally efficient predictions can be achieved without measurably increasing model complexity. Strong predictive performance should therefore not be viewed solely as the result of a single model choice, a specific evaluation procedure, or a feature construction strategy. Ultimately, successful tennis match outcome prediction depends on how these elements interact in a realistic forecasting setting.

References

- Angelini, G., Candila, V. & De Angelis, L. (2022). Weighted Elo rating for tennis match predictions. *European Journal of Operational Research* 297(1), pp. 120–132.
- Baumer, B., Matthews, G. & Nguyen, Q. (2023). Big ideas in sports analytics and statistical tools for their investigation. *Wiley Interdisciplinary Reviews: Computational Statistics* 15(6), e1612.
- Bozděch, M. & Zháněl, J. (2023). Analyzing game statistics and career trajectories of female elite junior tennis players: A machine learning approach. *PLoS One* 18(11), e0295075.
- Breiman, L. (2001). Random forests. *Machine Learning* 45(1), pp. 5–32.
- Breiman, L., Friedman, J., Olshen, R. & Stone, C. (1984). *Classification and Regression Trees*. Wadsworth International Group.
- Buhamra, N., Groll, A. & Brunner, S. (2024). Modeling and prediction of tennis matches at Grand Slam tournaments. *Journal of Sports Analytics* 10(1), pp. 17–33.
- Bunker, R. & Thabtah, F. (2019). A machine learning framework for sport result prediction. *Applied Computing and Informatics* 15(1), pp. 27–33.
- Bunker, R., Yeung, C., Susnjak, T., Espie, C. & Fujii, K. (2024). A comparative evaluation of Elo ratings- and machine learning-based methods for tennis match result prediction. *Proceedings of the Institution of Mechanical Engineers, Part P: Journal of Sports Engineering and Technology* 238(4), pp. 305–316.
- Cox, D. (1958). The regression analysis of binary sequences. *Journal of the Royal Statistical Society Series B: Statistical Methodology* 20(2), pp. 215–232.
- Elo, A. (1978). *The Rating of Chessplayers, Past and Present*. Second Edition. First edition published 1978; Second edition printed 1986. New York, USA: Arco Publishing, Inc.
- Gao, Z. & Kowalczyk, A. (2021). Random forest model identifies serve strength as a key predictor of tennis match outcome. *Journal of Sports Analytics* 7(4), pp. 255–262.
- Hastie, T., Tibshirani, R. & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer.
- Horvat, T. & Job, J. (2020). The use of machine learning in sport outcome prediction: A review. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 10(5), e1380.
- James, G., Witten, D., Hastie, T., Tibshirani, R. & Taylor, J. (2023). *An Introduction to Statistical Learning: with Applications in Python*. Springer.
- Klaassen, F. & Magnus, J. (2001). Are points in tennis independent and identically distributed? Evidence from a dynamic binary panel data model. *Journal of the American Statistical Association* 96(454), pp. 500–509.
- Kovalchik, S. (2020). Extension of the Elo rating system to margin of victory. *International Journal of Forecasting* 36(4), pp. 1329–1341.

- Ma, W., Wang, S., Venet, A., Gu, J., Subramanian, S., Liu, R., Yang, Y. & Friedman, D. (2024). F3: A Compiler for Feature Engineering. In: *Proceedings of the 2nd ACM SIGPLAN International Workshop on Functional Software Architecture*, pp. 3–9.
- Nayem, H., Aziz, S. & Kibria, B. (2025). Evaluating estimator performance under multicollinearity: A trade-off between MSE and accuracy in logistic, Lasso, elastic net, and ridge regression with varying penalty parameters. *Stats* 8(2), p. 45.
- Sackmann, J. (2019). *An Introduction to Tennis Elo*. [Website]. [Accessed: 2026-03-20]. <https://www.tennisabstract.com/blog/2019/12/03/an-introduction-to-tennis-elo/>.
- Sackmann, J. (2026). *ATP Tennis Rankings, Results, and Stats*. [GitHub repository]. [Accessed: 2026-05-14]. https://github.com/JeffSackmann/tennis_atp.
- scikit-learn (2023a). 3.4. *Metrics and scoring: quantifying the quality of predictions*. [Website]. [Accessed: 2026-03-20]. https://scikit-learn.org/stable/modules/model_evaluation.html.
- scikit-learn (2023b). *brier_score_loss*. [Website]. [Accessed: 2026-03-23]. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.brier_score_loss.html.
- scikit-learn (2023c). *log_loss*. [Website]. [Accessed: 2026-03-23]. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.log_loss.html.
- Sipko, M. (2015). *Machine learning for the prediction of professional tennis matches*. MEng final year project, Department of Computing, Imperial College London. Unpublished project report.
- Wilkens, S. (2021). Sports prediction and betting models in the machine learning age: The case of tennis. *Journal of Sports Analytics* 7(2), pp. 99–117.